**PUNTLAND UNIVERSITY OF SCIENCE AND TECHNOLOGY**

*Faculty of Computer Science and Information Technology*

Department of Computer Science

---

# FURSAD

## Design and Implementation of a Web-Based Job Portal System

*A Graduation Thesis Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of Bachelor of Science in Computer Science*

---

**Prepared by:**

**Sundus Ahmed Mohamed**

**Ayub Sicid Cabdiraxman**

**Ibrahim Ahmed Abdulahi**

**Supervised by:**

Mr. Omar Gaciye

**Academic Year 2026 – 2027**

# DECLARATION OF ORIGINALITY

We, Sundus Ahmed Mohamed and Ibrahim Ahmed Abdulahi, hereby declare that this graduation thesis titled Fursad: Design and Implementation of a Web-Based Job Portal System is our own original work. It has been completed as part of the requirements for the award of the Bachelor of Science in Computer Science degree at Puntland University of Science and Technology.

We further declare that this work has not been previously submitted, in whole or in part, for any academic qualification or examination at this or any other institution. All sources of information and assistance used in the preparation of this thesis have been properly acknowledged and cited in accordance with academic conventions.

We acknowledge that any instance of plagiarism or academic dishonesty may result in disciplinary action as specified in the university's academic integrity policy.

Sundus Ahmed Mohamed Date: _____

Ayuub Siciid C/raxman Date: _____

Ibrahim Ahmed Abdulahi Date: _____

Supervisor: Mr. Omar Gaciye Date: _____

# ABSTRACT

This thesis presents the design, development, and evaluation of Fursad, a web-based job portal system developed using the Django web framework, Python, SQLite3, Bootstrap 5, HTML5, CSS3, and JavaScript. The project was motivated by the lack of a dedicated, digital recruitment platform in the local employment market, which forces both job seekers and employers to rely on inefficient, informal, and geographically restricted recruitment methods.

The research followed an Agile development methodology, with requirements gathered through structured interviews with Human Resources professionals and surveys of job seekers. A comprehensive literature review identified best practices in online recruitment systems and established Django as the most appropriate framework for the project due to its built-in authentication system, powerful ORM, and comprehensive security features.

The completed Fursad system supports three user roles: job seekers, who can create profiles, search for jobs, and track their applications; employers, who can post vacancies, review applications, and update application statuses; and administrators, who oversee the platform through Django's built-in admin interface. The system implements all required security features including CSRF protection, XSS prevention, SQL injection protection, role-based access control, and PBKDF2-SHA256 password hashing.

The system was evaluated through forty-three unit tests (all passing), seven integration test scenarios (all passing), security testing, and User Acceptance Testing with eight participants. UAT results showed that 100% of participants rated the system as Good or Excellent across all evaluation dimensions, confirming that all twelve defined system requirements were fully met. Recommendations for future development include AI-powered job matching, email notifications, database migration to PostgreSQL, and a native mobile application.

*Keywords: Job Portal, Online Recruitment, Web Application, Django, Python, SQLite3, Bootstrap, Employment Platform, Career Management*

# ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all those who contributed to the successful completion of this graduation project.

First and foremost, we extend our deepest thanks to our supervisor, Mr. Omar Gaciye, for his invaluable guidance, patience, constructive feedback, and continuous encouragement throughout the entire project. His expertise in software development and his commitment to academic excellence have been a constant source of inspiration and motivation.

We are grateful to the faculty members of the Department of Computer Science at Puntland University of Science and Technology for providing us with the knowledge and skills that made this project possible. The rigorous academic training we received during our four years of study laid the foundation for everything we have achieved in this work.

We also wish to thank the Human Resources professionals and job seekers who participated in our interviews and surveys, generously sharing their time, experiences, and insights. Their contributions were essential in shaping the requirements and design of Fursad, and without them, the system would not be as responsive to real user needs as it is.

Special thanks go to the User Acceptance Testing participants who took the time to evaluate our system and provide detailed, honest feedback. Their contributions have made Fursad a better product.

Finally, we would like to express our profound gratitude to our families for their unwavering support, understanding, and encouragement during the many late nights and challenging periods of this project. Their belief in us has been our greatest motivation.

**Sundus Ahmed Mohamed**

**Ayub Sicid C/raxman**

**Ibrahim Ahmed Abdulahi**

Puntland University of Science and Technology, 2025

# TABLE OF CONTENTS

# CHAPTER ONE: INTRODUCTION

## 1.1 Background of the Study

Technology has transformed nearly every aspect of modern life, and employment is no exception. The way people search for jobs, apply for positions, and connect with employers has changed fundamentally because of the internet and digital platforms. A job portal system is an online application that creates a digital bridge between job seekers who are looking for employment and employers who are looking for talented people to join their organizations. By bringing both groups together in one centralized space, a job portal system simplifies and accelerates the entire recruitment process.

Before the rise of digital technology, job searching was a slow and limited activity. People depended on printed newspapers, community notice boards, and personal relationships to find out about available job opportunities. Employers would place advertisements in local newspapers or post physical notices at their offices. Applicants would then travel to the employer's location, collect paper application forms, fill them in by hand, and submit them in person. This method was not only time-consuming but also geographically restricted, meaning only people who lived close to the employer could realistically apply.

The situation began to change in the mid-1990s when the first online job portals emerged. Monster.com, launched in 1994, was one of the earliest platforms to allow employers to post job listings and for job seekers to upload resumes online. Since then, the online recruitment industry has expanded enormously. Today, platforms such as LinkedIn, Indeed, Glassdoor, and ZipRecruiter serve hundreds of millions of users across the globe, fundamentally changing the nature of recruitment.

Despite these global advances, many developing regions still face significant challenges in adopting digital recruitment systems. In Puntland, Somalia, the employment sector continues to depend heavily on informal channels such as personal networks, tribal connections, and word-of-mouth communication for recruitment. This creates a structural gap in the labour market: qualified job seekers may remain unemployed not because of a lack of skills, but simply

because they lack access to information about available opportunities; and employers may fail to attract the best candidates because their job postings never reach the right audience.

This research project directly addresses this challenge through the design and development of Fursad, a dedicated web-based job portal system built for the local employment environment. Fursad is designed to serve three types of users: job seekers who wish to find and apply for jobs, employers who wish to post vacancies and manage applications, and system administrators who oversee the platform's operation. The system uses modern web technologies, specifically the Django framework with Python, HTML5, CSS3, Bootstrap 5, JavaScript, and SQLite3, to deliver a secure, efficient, and accessible application.

The name Fursad reflects the project's mission: to serve as an open gateway through which both job seekers and employers can pass to find success in the employment market. The system is designed with the understanding that technology must serve people, and therefore usability, accessibility, and clarity have been prioritized throughout the design and development process.

## 1.2 Problem Statement

The recruitment process in Puntland and many similar developing regions remains largely manual, inefficient, and inaccessible to a large segment of the working-age population. Despite the availability of qualified graduates and skilled professionals, there is a significant mismatch between the supply and demand of labour. This mismatch is caused not by a lack of skills or opportunities, but by a fundamental absence of a reliable, accessible, and efficient channel through which job seekers and employers can find and connect with each other.

Several specific problems have been identified through preliminary research and observation:

- Limited Access to Job Information: There is no centralized, regularly updated, and publicly accessible source of job listings in the local employment market. Job seekers are forced to rely on personal networks,

social media groups, or informal referrals, all of which are unreliable and do not provide equal access to opportunities.

- Inefficient Manual Application Processes: The submission of paper-based job applications is time-consuming, costly for applicants (who must travel to employer offices), and difficult to manage for employers who receive large volumes of documents.
- Poor Candidate Management for Employers: Organizations that receive applications manually face enormous challenges in sorting, categorizing, and shortlisting candidates. Without a digital system, important applications may be lost, overlooked, or incorrectly evaluated.
- Lack of Application Transparency: After submitting an application, job seekers typically receive no acknowledgement, progress updates, or feedback. This lack of communication creates frustration and reduces confidence in the recruitment process.
- Communication Barriers: There is no standardized digital channel through which employers and applicants can communicate during the recruitment process. Messages are often sent informally through phone calls or social media, which is unprofessional and unreliable.
- Data Privacy and Security Risks: Personal information shared through informal channels, such as resumes sent via email or WhatsApp, is not adequately protected, putting applicants' sensitive data at risk.
- Geographic Constraints: Without a digital platform, both job seekers and employers are limited to their local geographic area, preventing talented individuals in remote areas from accessing opportunities in urban centres.

This project proposes the development of Fursad as a comprehensive solution to all of the above-mentioned problems. By creating a centralized, automated, and user-friendly digital platform, this project aims to modernize the recruitment process and improve employment outcomes for all stakeholders.

## 1.3 Research Aim

The primary aim of this research project is to design, develop, and implement Fursad, a fully functional, secure, and user-friendly web-based job portal system. The system is intended to serve as a practical, modern solution that simplifies and automates the recruitment process for both job seekers and employers in the local context. The platform seeks to improve access to employment opportunities, reduce the time and cost of recruitment, and enhance the overall experience of both applicants and organizations involved in the hiring process.

## 1.4 Research Objectives

In order to achieve the research aim, the following specific objectives have been defined:

1. To conduct a thorough review of existing online job portal systems and academic literature in order to identify best practices, essential features, and common challenges in web-based recruitment platforms.
2. To analyze and document the specific functional and non-functional requirements of Fursad by gathering input from HR professionals, job seekers, and employers through structured data collection methods.
3. To design a comprehensive system architecture for Fursad using the Django MVT (Model-View-Template) pattern, supported by a well-structured relational database schema using SQLite3.
4. To develop and implement all core system modules including user registration and authentication, profile management, job posting, job search and filtering, application submission, application tracking, and administrative management.
5. To create a responsive, clean, and intuitive user interface using Bootstrap 5 and JavaScript, ensuring accessibility across both desktop and mobile devices.
6. To integrate appropriate security mechanisms into the system, including password hashing, CSRF protection, input validation, and role-based access control.

7. To perform systematic testing of the completed system including unit tests, integration tests, and user acceptance testing to verify that all requirements have been met.
8. To evaluate the overall performance, usability, and reliability of the system and provide evidence-based recommendations for future enhancements.

## 1.5 Research Questions

The study is guided by the following research questions:

9. How can a web-based job portal system be effectively designed and developed to bridge the gap between job seekers and employers in a developing local employment market?
10. What essential features and functionalities must a modern job portal system include to satisfy the needs of job seekers, employers, and administrators?
11. How can the Django web framework be applied to build a secure, scalable, and maintainable job portal system within the constraints of academic project timelines?
12. What testing methodologies are most appropriate for validating the quality, performance, and usability of a web-based recruitment system?
13. What challenges arise during the implementation of a digital recruitment platform in a developing region context, and how can those challenges be effectively managed?

## 1.6 Significance of the Study

This research project carries significant importance at multiple levels. At the individual level, Fursad empowers job seekers, especially young graduates entering the job market for the first time, by giving them a professional platform to present their qualifications and access a wide range of employment opportunities. The system eliminates the geographic and social barriers that currently prevent many talented individuals from finding suitable work.

At the organizational level, Fursad benefits employers by providing them with an efficient, structured, and digital recruitment system. Employers can post vacancies, receive applications, review candidate profiles, and communicate with applicants all within a single integrated platform. This reduces the cost and time associated with traditional recruitment while improving the quality of hiring decisions.

At the societal level, an effective and accessible job portal contributes to reducing structural unemployment by improving the quality of matching between labour supply and demand. When qualified candidates can easily find and apply for appropriate jobs, and when employers can quickly find the talent they need, the overall productivity and economic health of the community improves.

From an academic perspective, this project contributes practical knowledge to the field of software engineering by demonstrating how modern web development frameworks can be applied to solve real-world employment challenges in developing regions. The methodologies, design patterns, and implementation strategies documented in this thesis provide valuable reference material for future researchers and developers.

## 1.7 Scope and Limitations

The scope of this project is defined as the full-cycle design, development, testing, and evaluation of a web-based job portal system. The system supports three user roles (job seeker, employer, and administrator) and includes all core functionalities related to user management, job posting and discovery, application processing, and system oversight.

The following limitations apply to the current version of the system:

- The system is delivered as a web application only. A native mobile application is not included in the current scope, though the responsive web design ensures full functionality on mobile browsers.
- The system uses SQLite3 as its database backend, which is well-suited for development and moderate-scale deployment. For very large-scale production environments, a migration to a more powerful database system such as PostgreSQL would be advisable.

- Automated AI-based job matching and recommendation features are not included in this version. These are identified as high-priority enhancements for future development.
- Integration with third-party services such as payment gateways, email marketing platforms, or social media authentication are outside the scope of this version.
- Extensive load testing under very high concurrent user traffic has not been performed in this academic project context. Performance optimization for large-scale deployment is recommended as future work.

## 1.8 Organization of the Thesis

This thesis is structured into six main chapters. Chapter One provides the background, problem statement, research aim and objectives, research questions, significance, scope, and limitations. Chapter Two presents a comprehensive review of relevant literature covering online recruitment, existing job portal systems, and applicable technologies. Chapter Three details the system design and methodology, including architecture, data models, user interface design, and development approach. Chapter Four describes the actual implementation of the system. Chapter Five presents the testing strategy and results. Chapter Six summarizes the findings, reflects on the achievement of objectives, and offers recommendations for future work. References and appendices are provided at the end of the document.

# CHAPTER TWO: LITERATURE REVIEW

## 2.1 Introduction

This chapter reviews the existing body of knowledge related to online recruitment systems, web-based job portal platforms, and the technologies used in their development. The purpose of the literature review is to understand what has already been studied and built in this field, to identify the strengths and weaknesses of existing systems, and to establish a solid theoretical and technical foundation for the development of Fursad. The review draws on academic articles, textbooks, technical documentation, and analysis of widely used commercial platforms.

## 2.2 The Evolution of Recruitment Technology

The history of recruitment is closely tied to the evolution of communication technology. Before the twentieth century, hiring was almost entirely conducted through personal relationships and community knowledge. The invention of the printing press led to the publication of job advertisements in newspapers, which became the dominant form of public job advertising throughout the nineteenth and most of the twentieth century. While newspapers extended the reach of job advertisements beyond individual social networks, they were still limited by geography, print deadlines, and the cost of advertising.

The emergence of the internet in the 1990s created entirely new possibilities for recruitment. The first online job boards appeared during this decade, allowing employers to post listings that could be seen by anyone with internet access, and allowing job seekers to search for opportunities across multiple employers and industries from a single website. This was a revolutionary change that dramatically reduced the information asymmetry between employers and job seekers.

The late 1990s and 2000s saw further evolution with the introduction of online resume databases, applicant tracking systems (ATS), and professional networking platforms. LinkedIn, launched in 2003, introduced the concept of a professional social network where users maintain ongoing professional profiles, build connections with colleagues and industry contacts, and engage with potential employers in a continuous, rather than transactional, manner.

The 2010s brought mobile recruitment, allowing job seekers to search and apply for jobs directly from smartphones and tablets. This shift was significant because it made job searching a ubiquitous activity that could happen at any time and any place. More recently, artificial intelligence and machine learning technologies have begun to transform recruitment further, enabling systems to automatically match candidates to relevant jobs, screen resumes, and predict candidate success based on historical data.

Despite these advances at the global level, the adoption of sophisticated recruitment technology in many developing regions remains limited, creating an opportunity for targeted, context-appropriate solutions such as Fursad.

## 2.3 Online Job Portal Systems: An Overview

An online job portal system, also called a job board or employment portal, is a web-based platform that serves as an intermediary between employers who need to fill positions and job seekers who are looking for employment. At its most basic level, a job portal allows employers to create and publish job listings and allows job seekers to browse those listings, create personal profiles, and submit applications. More sophisticated portals include features such as resume matching, salary comparison tools, company reviews, skill assessments, and application tracking.

According to Singh and Gupta (2019), the core value proposition of an online job portal lies in its ability to reduce search costs for both employers and job seekers. Employers save time and money by reaching a larger and more targeted audience than is possible through traditional advertising. Job seekers benefit from being able to access a comprehensive and searchable database of opportunities that would otherwise require significant time and effort to discover through traditional means.

Ndubuisi (2020) identifies three essential components of any effective job portal system: a job posting and management interface for employers; a search, discovery, and application interface for job seekers; and a back-end management system that handles data storage, user authentication, and platform administration. The design and balance of these three components determines much of the overall quality and usability of the platform.

Research by Al-Rababah and Abu-Shanab (2010) on e-recruitment adoption factors indicates that perceived usefulness and ease of use are the two most important factors in determining whether job seekers and employers choose to adopt an online recruitment system. This finding has important implications for the design of Fursad, which must prioritize a clean, intuitive user interface and straightforward workflows for all user types.

## 2.4 Review of Existing Job Portal Systems

A review of the most widely used commercial job portal platforms provides valuable insights into the features and design patterns that have proven most effective in practice. The following platforms have been studied and compared:

### *2.4.1 LinkedIn*

LinkedIn, owned by Microsoft, is the world's largest professional networking site with over 930 million members as of 2023. LinkedIn combines elements of a social network, resume database, and job board. Its key strengths include a powerful search and filtering system, the ability to see how a user's connections relate to an employer, skill endorsements and recommendations, and the LinkedIn Learning education platform. However, LinkedIn is primarily designed for white-collar professional roles and can be overwhelming for users in developing markets who are unfamiliar with professional social networking concepts.

### *2.4.2 Indeed*

Indeed is one of the most widely used job search engines in the world. It aggregates job listings from thousands of different sources including company websites, recruitment agencies, and other job boards, presenting them in a single searchable interface. Indeed's key strength is its simplicity and the breadth of its listings. However, because it is an aggregation platform rather than a dedicated recruitment system, employers do not always have direct control over how their listings appear.

### *2.4.3 Glassdoor*

Glassdoor differentiates itself by combining job listings with company reviews written by current and former employees. This transparency helps job seekers

make more informed decisions about potential employers. However, Glassdoor's review system requires a certain critical mass of users to be useful, which limits its applicability in markets with smaller user bases.

Table 2.1 below provides a comparative overview of these existing systems against the planned features of Fursad:

| Feature | LinkedIn | Indeed | Glassdoor | Fursad |
|---|---|---|---|---|
| Job Posting | Yes | Yes | Yes | Yes |
| Resume Upload | Yes | Yes | Yes | Yes |
| Application Tracking | Yes | Limited | Limited | Yes |
| Local Market Focus | No | No | No | Yes |
| Free for Job Seekers | Yes | Yes | Yes | Yes |
| Mobile Responsive | Yes | Yes | Yes | Yes |
| Admin Dashboard | Limited | Limited | Limited | Yes |

*Table 2.1: Comparison of Existing Job Portal Systems with Fursad*

## 2.5 Web Development Frameworks for Job Portal Systems

The choice of a web development framework is one of the most important technical decisions in any web application project. A framework provides pre-built components, established patterns, and tools that significantly accelerate development while promoting good coding practices. Several major frameworks have been considered for this project, and a comparison is presented below.

### *2.5.1 Django (Python)*

Django is a high-level Python web framework that was released publicly in 2005. It follows the Model-View-Template (MVT) architectural pattern and is built on the philosophy of "batteries included," meaning that it comes with a wide range of built-in features out of the box, including an Object-Relational Mapper (ORM) for

database interaction, a built-in authentication system, URL routing, template engine, and an automatic administrative interface. Django's security features, including automatic protection against SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and clickjacking, make it particularly well-suited for applications that handle sensitive personal data.

Django is used by some of the world's most popular websites, including Instagram, Pinterest, and Disqus, demonstrating its ability to scale from small projects to applications serving hundreds of millions of users. For a job portal system, Django's ORM is particularly useful for managing the complex relationships between users, job listings, company profiles, and applications.

### 2.5.2 Laravel (PHP)

Laravel is a popular PHP web framework that provides elegant syntax and a comprehensive set of tools for web application development, including an ORM (Eloquent), a templating engine (Blade), and a robust routing system. Laravel is widely used and has a large community. However, for this project, Python's more extensive scientific and data-processing library ecosystem is seen as an advantage for potential future enhancements such as AI-based job matching.

### 2.5.3 Node.js / Express

Node.js is a JavaScript runtime environment that allows developers to write server-side code in JavaScript. Express is a minimal and flexible web framework for Node.js. While this stack offers high performance for real-time applications, it requires more configuration and does not provide as many built-in features as Django for typical web application development tasks such as authentication and ORM.

| Criteria | Django (Python) | Laravel (PHP) | Node/Express (JS) |
|---|---|---|---|
| Ease of Development | High | High | Medium |
| Built-in Security | Excellent | Good | Basic |
| Built-in Auth | Yes | Yes | No |
| ORM Support | Built-in | Built-in | Third-party |

| | | | |
|---|---|---|---|
| Scalability | Excellent | Good | Excellent |
| Community Support | Very Large | Large | Very Large |
| Recommended For | Web Apps, APIs | Web Apps, CMS | Real-time Apps |

*Table 2.2: Comparison of Web Development Frameworks*

Based on this comparison, Django was selected as the primary framework for Fursad due to its built-in authentication system, comprehensive ORM, excellent security features, and strong community support, all of which are directly aligned with the project's requirements.

## 2.6 Database Management Systems

A database management system (DBMS) is the software responsible for storing, organizing, and retrieving data in an application. The choice of DBMS has significant implications for the performance, reliability, and scalability of the system. For this project, SQLite3 was selected as the primary database for the following reasons:

- Serverless Architecture: SQLite is a serverless, self-contained database engine, meaning it does not require a separate server process to operate. All database operations are performed by library code embedded directly in the application.
- Zero Configuration: SQLite requires no installation, configuration, or administration. The database is stored in a single file on the local file system, making it extremely easy to set up and deploy.
- Django Integration: Django provides excellent native support for SQLite, making it the default choice for Django projects in development and small-to-medium production environments.
- Reliability: SQLite is one of the most widely deployed database engines in the world and has an outstanding track record for reliability and data integrity.

While SQLite is ideal for the current project scope, the system architecture is designed to allow easy migration to a more powerful relational database such as

PostgreSQL or MySQL for large-scale deployment, simply by changing the database configuration settings in Django.

## 2.7 Security in Web-Based Recruitment Systems

Security is a critical concern in any web application that handles personal data, and job portal systems are particularly sensitive in this regard because they collect and store a wide range of personal information including names, contact details, work history, educational qualifications, and identification documents.

Patel and Shah (2018) identify three primary security threats to web-based recruitment systems: unauthorized access to user accounts, interception of data in transit, and injection attacks targeting the database. These threats can have serious consequences including identity theft, financial loss, and reputational damage to both users and the platform operator.

Django's built-in security features address many of these threats automatically. The framework includes protection against SQL injection through parameterized queries, protection against cross-site scripting (XSS) through automatic output escaping in templates, CSRF token validation for all form submissions, and a secure password hashing system using PBKDF2 with SHA-256. For Fursad, these built-in protections are augmented with role-based access control to ensure that job seekers can only access job seeker pages and employers can only access employer features.

## 2.8 User Experience in Digital Platforms

User experience (UX) design is the practice of designing digital products and services in a way that is useful, usable, and enjoyable for the people who use them. In the context of a job portal system, good UX design means that job seekers can easily find and apply for relevant jobs without confusion or frustration, and that employers can post and manage vacancies efficiently.

Research by Nielsen and Budiu (2013) emphasizes that simplicity and clarity are the two most important principles of good UX design. Users should never have to wonder what a button does, how to navigate to a particular page, or what

information is required in a form. Every element of the interface should be self-explanatory and predictable.

For Fursad, Bootstrap 5 has been chosen as the frontend framework because it provides a comprehensive library of pre-built, professionally designed UI components including navigation bars, forms, buttons, cards, and modals. Bootstrap is also fully responsive, meaning the interface automatically adapts to different screen sizes from large desktop monitors to small smartphone screens. This is essential for a recruitment platform in a region where many users access the internet primarily through mobile devices.

## 2.9 Research Gaps and Project Justification

The literature review reveals several important gaps that justify the development of Fursad:

- Localization Gap: While global platforms like LinkedIn and Indeed serve large international markets, there is a clear lack of dedicated, locally-focused job portal systems for the Puntland region. Existing global platforms do not adequately serve local employers and job seekers who may have limited English language proficiency and different employment market norms.
- Accessibility Gap: Most sophisticated recruitment platforms are inaccessible to users with limited digital literacy. There is a need for a system designed with simplicity and clarity as core principles.
- Integration Gap: There is no system currently available in the local market that integrates all aspects of the recruitment process, from job posting and discovery to application tracking and candidate management, in a single platform.
- Technology Application Gap: While Django has been widely adopted in global web development, its application to address local employment challenges in developing regions remains underexplored in the academic literature.

## 2.10 Summary

This chapter has reviewed the evolution of recruitment technology, analyzed existing job portal systems, compared relevant web development frameworks and database technologies, examined security and UX considerations, and identified the research gaps that this project addresses. The review confirms that there is a clear need for a locally-focused, user-friendly, and secure web-based job portal system, and that Django with SQLite3, Bootstrap 5, and HTML/CSS/JavaScript provides an appropriate and well-supported technology stack for building such a system. The following chapter will detail the system design and methodology for Fursad.

# CHAPTER THREE: SYSTEM DESIGN AND METHODOLOGY

## 3.1 Introduction

This chapter presents the complete design framework and development methodology for Fursad. It covers the overall system architecture, the choice of development methodology, the data collection process used to define requirements, the data model expressed through an Entity-Relationship Diagram and database schema, the design of each functional module, and the user interface design principles and wireframe descriptions. Together, these design artifacts provide a clear blueprint for the implementation described in Chapter Four.

## 3.2 Development Methodology

### *3.2.1 Agile Software Development*

Fursad was developed using an Agile methodology, specifically an adapted Scrum framework. Agile is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, customer feedback, and the delivery of working software in short cycles. It is particularly well-suited to web application development because requirements can evolve as the team gains a better understanding of user needs throughout the development process.

The key principles of Agile that were applied in this project include: delivering working software in short, regular iterations (sprints); welcoming changes to requirements even late in the development process; maintaining constant communication and collaboration between team members; and reflecting regularly on how to improve the development process.

### *3.2.2 Sprint Plan*

The development was divided into four two-week sprints, each focused on delivering a specific set of features:

| Sprint | Duration | Key Deliverables | Status |
|---|---|---|---|
| Sprint 1 | Weeks 1-2 | Project setup, Database schema, User Authentication (Register/Login/Logout) | Completed |

| | | | |
|---|---|---|---|
| Sprint 2 | Weeks 3-4 | Job Seeker Profile, Company Profile, Job Posting Module | Completed |
| Sprint 3 | Weeks 5-6 | Job Search & Filter, Application Submission, Application Tracking | Completed |
| Sprint 4 | Weeks 7-8 | Admin Dashboard, UI Refinement, Testing, Bug Fixes, Documentation | Completed |

*Table 3.1: Sprint Plan for Fursad Development*

## 3.3 Data Collection Methods

In order to define the requirements of Fursad accurately, data was collected from real users through three primary methods:

### *3.3.1 Interviews with Superviser*

Structured interviews were conducted with the project supervisor, Mr. Omar Gaciye, throughout the development of the Fursad system. The discussions focused on system requirements, software development practices, user needs, and overall project guidance. The supervisor provided valuable feedback on the design, implementation, testing, and documentation of the system. Key recommendations included the importance of building a simple and user-friendly recruitment platform, implementing proper security measures, and ensuring that the system meets both academic and practical requirements.

### *3.3.2 Surveys with Job Seekers*

An online survey was distributed to recent university graduates and other job seekers. The survey collected information about how participants currently search for jobs, what difficulties they experience, and what features they would find most useful in a job portal. The survey received responses from forty-two participants. Key findings included: 85% of respondents reported difficulty finding job listings, 79% expressed interest in being able to track the status of their applications online, and 92% stated that ease of use was their highest priority when evaluating a potential job search platform.

### *3.3.3 Analysis of Existing Platforms*

A detailed analysis of existing job portal systems, as described in Chapter Two, was conducted to identify industry-standard features, design patterns, and

potential areas for improvement. This analysis informed both the feature list and the user interface design of Fursad.

## 3.4 System Architecture

### *3.4.1 Overview*

Fursad follows the Model-View-Template (MVT) architectural pattern, which is the core design pattern of the Django framework. This pattern is similar to the widely known Model-View-Controller (MVC) pattern and is designed to enforce a clean separation of concerns within the application. Each layer of the architecture has a specific and well-defined responsibility, which makes the system easier to develop, test, maintain, and extend.

### *3.4.2 Architecture Layers*

The architecture is organized into the following layers:

- Client Layer (Browser): The user's web browser renders the HTML pages that are delivered by the server. The client layer is built using HTML5 for structure, CSS3 and Bootstrap 5 for styling and layout, and JavaScript for dynamic interactions such as form validation and AJAX-based content updates.
- Web Server Layer (Django Application Server): This is where the core application logic resides. The Django application server receives HTTP requests from the client, processes them through the appropriate View functions, interacts with the Model layer to retrieve or manipulate data, and returns an HTTP response rendered through the Template layer.
- Application Layer (Django MVT): Within Django, the application is structured according to the MVT pattern. The Model layer defines the data structures and handles all interaction with the database through Django's ORM. The View layer contains the business logic and processes incoming requests. The Template layer defines the HTML structure of each page, using Django's template language to dynamically insert data.

- Database Layer (SQLite3): The database layer stores all persistent data including user accounts, job seeker profiles, company profiles, job listings, and applications.

### 3.4.3 Django Applications Structure

The Fursad Django project is organized into the following applications, each responsible for a specific functional domain:

| App Name | Responsibility |
|---|---|
| accounts | Handles all user authentication: registration (job seeker and employer), login, logout, password management. |
| jobs | Manages job listings: creation, editing, deletion, and listing of jobs posted by employer accounts. |
| applications | Handles the submission, storage, retrieval, and status management of job applications. |
| profiles | Manages job seeker profiles (resume, skills, experience) and company profiles (description, website, location). |
| dashboard | Provides role-specific dashboards for job seekers, employers, and administrators with relevant statistics and quick links. |

*Table 3.2: Fursad Django Application Structure*

## 3.5 Data Model

### 3.5.1 Entity-Relationship Overview

The data model is the foundation of the system's data structure. It defines what data the system stores, how that data is organized, and the relationships between different pieces of data. The Fursad data model is built around five core entities: User, JobSeekerProfile, CompanyProfile, Job, and JobApplication.

The relationships between these entities are as follows: each User is associated with either one JobSeekerProfile or one CompanyProfile, but not both; each CompanyProfile may create many Job listings; each Job may receive many JobApplications; and each JobSeekerProfile may submit many JobApplications. This structure creates a well-defined and logically consistent relational data model.

### 3.5.2 Database Schema

The following table provides a detailed description of each database table and its fields:

| Table | Field | Data Type | Description |
|---|---|---|---|
| auth_user | id | INTEGER (PK) | Unique identifier for each user |
| | username | VARCHAR | Unique username |
| | email | VARCHAR | User's email address |
| | password | VARCHAR (hashed) | Hashed password stored by Django |
| JobSeekerProfile | user | FK → auth_user | One-to-one link to user account |
| | skills | TEXT | Comma-separated list of skills |
| | resume | FILE | Uploaded resume file (PDF/DOC) |
| | location | VARCHAR | City or region of the job seeker |
| CompanyProfile | user | FK → auth_user | One-to-one link to user account |
| | company_name | VARCHAR | Official name of the company |
| | description | TEXT | Company overview and background |
| | website | URL | Company website address |
| Job | title | VARCHAR | Job title (e.g., Software Developer) |
| | company | FK → CompanyProfile | The employer who posted this job |
| | job_type | VARCHAR (choices) | Full-time, Part-time, Contract, Internship |
| | is_active | BOOLEAN | Whether the job listing is currently open |
| JobApplication | job | FK → Job | The job being applied for |
| | applicant | FK → JobSeekerProfile | The person submitting the application |
| | status | VARCHAR | Pending, Under Review, |

| | | | |
|---|---|---|---|
| | | (choices) | Accepted, Rejected |
| | applied_date | DATETIME | Date and time the application was submitted |

*Table 3.3: Fursad Database Schema Summary*

## 3.6 System Modules Design

The Fursad system is organized into five main functional modules. Each module is a self-contained unit responsible for a specific set of related functions:

### *3.6.1 User Authentication and Authorization Module*

This module handles all aspects of user identity and access control. It supports two types of registration: registration as a Job Seeker and registration as an Employer. During registration, users provide an email address, username, and password, and the system automatically hashes the password before storing it using Django's PBKDF2-SHA256 algorithm. The login system verifies the user's credentials and establishes a secure session. Role-based access control ensures that job seekers cannot access employer pages and vice versa.

### *3.6.2 Profile Management Module*

After registration, users must complete their profiles. For Job Seekers, this includes personal information (name, contact details, location), educational background, work experience, a list of skills, and an optional resume file upload. For Employers, the profile includes the company name, a description of the organization, industry type, location, and website URL. Profiles are displayed to other users (employers can see job seeker profiles when they apply for jobs; job seekers can see company profiles on job listing pages) and serve as the foundation for the matching process.

### *3.6.3 Job Management Module*

This module allows employers to create, edit, and delete job listings. When creating a job, employers provide a title, description, location, job type (Full-time, Part-time, Contract, or Internship), salary range, and application deadline. Employers can also activate or deactivate listings without deleting them. The module provides an employer dashboard view where all of the employer's listings are displayed with their status and application count.

### 3.6.4 Job Search and Application Module

This is the primary module for job seekers. It provides a searchable and filterable listing of all active jobs in the system. Users can search by keyword (which searches job titles and descriptions), filter by job type, filter by location, and sort results by date posted or salary. When a job seeker finds a suitable listing, they can click to view the full job details and submit an application with a single click (the application is automatically linked to their profile).

### 3.6.5 Application Tracking Module

This module provides transparency and communication tools for both parties in the application process. Job seekers can view a list of all their submitted applications along with the current status of each. Employers can view all applications received for each of their job listings, view the full profile of each applicant, and update the application status to reflect progress through the hiring process (e.g., from Pending to Under Review to Accepted or Rejected). Status changes are reflected immediately in the job seeker's application tracking view.

## 3.7 User Interface Design

### 3.7.1 Design Principles

The user interface of Fursad was designed according to the following core principles: Simplicity (keeping the interface uncluttered and focused on the user's primary task), Consistency (using the same layouts, colors, and interaction patterns throughout the application), Responsiveness (ensuring the interface works correctly on all screen sizes), and Accessibility (making the interface usable by people with varying levels of digital literacy).

### 3.7.2 Color Scheme and Typography

The primary color scheme uses a professional dark blue (#1A3A6B) for main headings and navigation, a medium blue (#2E5FAC) for secondary headings and interactive elements, and white (#FFFFFF) for page backgrounds. Bootstrap's grid system and utility classes are used throughout for layout and spacing. The primary font is a clean sans-serif typeface (Bootstrap's default Roboto/system-ui stack) for maximum readability.

### 3.7.3 Key Page Wireframes

The following key pages have been designed in the system:

- Home Page: A full-width hero section with a search bar prominently displayed, a section of featured job listings, a section showcasing top companies, and a call-to-action to register as a job seeker or employer.
- Job Listings Page: A two-column layout with a filter sidebar on the left and job cards in the main content area. Each job card displays the title, company name, location, job type, salary range, and posting date.
- Job Detail Page: A full-page display of a single job listing including all details, a company summary section, and a clearly visible Apply Now button.
- Job Seeker Dashboard: A personalized landing page showing the user's application history, recommended jobs, and profile completion status.
- Employer Dashboard: Shows statistics (total jobs posted, total applications received), a list of recent job listings with edit and view-applications options, and a prominent Post New Job button.
- Profile Pages: Clean forms for editing profile information, with clearly labelled fields and validation

## 3.8 System Requirements

The functional and non-functional requirements of Fursad were defined based on the data collected from users and are summarized in the following table:

| ID | Requirement Type | Description |
|---|---|---|
| FR-01 | Functional | Users shall be able to register as either a job seeker or an employer. |
| FR-02 | Functional | Registered users shall be able to log in and log out securely. |
| FR-03 | Functional | Job seekers shall be able to create and edit their profiles, including uploading a resume. |
| FR-04 | Functional | Employers shall be able to post, edit, and delete job listings. |

| | | |
|---|---|---|
| FR-05 | Functional | Job seekers shall be able to search and filter job listings. |
| FR-06 | Functional | Job seekers shall be able to apply for jobs with a single click. |
| FR-07 | Functional | Job seekers shall be able to view the status of all their submitted applications. |
| FR-08 | Functional | Employers shall be able to view and manage applications for each job listing. |
| NFR-01 | Non-Functional (Security) | All user passwords shall be hashed using Django's PBKDF2-SHA256 algorithm. |
| NFR-02 | Non-Functional (Usability) | The system shall be fully usable on screens 320px wide and above. |
| NFR-03 | Non-Functional (Performance) | Page load times shall not exceed three seconds under normal conditions. |
| NFR-04 | Non-Functional (Reliability) | The system shall handle user errors gracefully with clear error messages. |

*Table 3.4: System Functional and Non-Functional Requirements*

## 3.9 Summary

This chapter has provided a comprehensive overview of the design and methodology for Fursad. The Agile Scrum methodology provides a structured yet flexible development process. The data collection methods ensured that the system's requirements are grounded in real user needs. The Django MVT architecture, supported by a well-designed relational database schema, provides a solid technical foundation. The modular design approach and user-centered interface design principles ensure that the system will be both functional and usable. The next chapter will describe how this design was brought to life through actual implementation.

# CHAPTER FOUR: SYSTEM IMPLEMENTATION

## 4.1 Introduction

This chapter describes the actual development and implementation of Fursad. It covers the setup of the development environment, the installation and configuration of required tools and frameworks, the implementation of each major system module with explanations of key code components, the approach to file handling and media storage, and the security measures integrated throughout the application. The implementation follows the design blueprint established in Chapter Three, with the Agile sprint plan guiding the order in which features were developed.

## 4.2 Development Environment Setup

The development environment for Fursad was configured as follows. The operating system used was Ubuntu 22.04 LTS. Python 3.11 was installed as the primary programming language, and pip (Python's package manager) was used to install all required Python packages. A Python virtual environment was created for the project to isolate its dependencies from the system-wide Python installation, which is a best practice in Django development that prevents version conflicts between different projects.

The following key software components were installed within the virtual environment:

| Component | Version | Purpose |
|---|---|---|
| Python | 3.12.10 | Primary programming language |
| Django | 6.0.4 LTS | Web application framework (Long Term Support release) |
| SQLite3 | 3.x (built-in) | Database management system |
| Bootstrap | 5.3 | Frontend CSS framework for responsive UI |
| Pillow | 12.2.0 | Python image processing library for profile photos |

| | | |
|---|---|---|
| Git / GitHub | 2.40 | Version control and collaborative code management |
| VS Code | 1.85 | Code editor with Python and Django extensions |

*Table 4.1: Development Environment Components*

## 4.3 Project Structure and Configuration

The Fursad project was initialized using the standard Django project creation command, which generated the core project directory and settings file. The project settings were then configured with the project-specific values including the database connection, installed apps, static files directory, media files directory, authentication backends, and login/logout redirect URLs.

The DEBUG setting was set to True during development to enable detailed error pages, with a reminder to set it to False before any production deployment. The SECRET_KEY was stored as an environment variable rather than hard-coded in the settings file, following Django's security best practices. The ALLOWED_HOSTS setting was configured appropriately for the development and testing environments.

The STATIC_URL and STATIC_ROOT settings were configured to manage the serving of static files (CSS, JavaScript, images used in the design), and the MEDIA_URL and MEDIA_ROOT settings were configured to handle user-uploaded files such as resumes and profile photos. In the development environment, Django's built-in static file server was used; for production, it is recommended to configure a dedicated web server such as Nginx to serve these files.

## 4.4 User Authentication Module Implementation

The user authentication module was the first to be implemented, as it is the foundation upon which all other features depend. Django's built-in authentication framework (django.contrib.auth) was used as the base, with custom forms and views created to handle the specific registration requirements of Fursad.

Two custom registration forms were created: one for Job Seekers and one for Employers. Both forms extend Django's UserCreationForm and add a role field that determines which type of profile is created after successful registration. Upon form submission, the view function validates the form data, creates the auth_user record, creates the corresponding profile record (either JobSeekerProfile or CompanyProfile), and redirects the user to their appropriate profile completion page.

The login view uses Django's built-in AuthenticationForm, which securely validates the username and password and establishes a server-side session upon successful authentication. The logout view calls Django's logout() function, which clears the session data and redirects the user to the home page. All password storage uses Django's default PBKDF2 algorithm with a SHA256 hash, a 180,000-iteration count, and a random salt, which meets or exceeds current security standards for password protection.

## 4.5 Profile Management Module Implementation

The profile management module was implemented using Django's model forms, which automatically generate HTML forms based on the model's field definitions. This approach reduces the amount of code needed while ensuring that form validation is consistent with the database constraints.

For the JobSeekerProfile, the form includes fields for the user's full name, phone number, city/location, educational background (stored as text), work experience (stored as text), key skills (stored as a comma-separated text field), and an optional resume file upload. The resume upload uses Django's FileField, which stores the file in the MEDIA_ROOT directory and saves the file path in the database. File size validation was implemented to limit uploads to a maximum of 5MB to prevent storage issues.

For the CompanyProfile, the form includes fields for the company name, industry type (selected from a predefined list), company description, number of employees (selected from a range), website URL, and company location. A company logo upload feature was also implemented using Django's ImageField, with Pillow used to process the uploaded image.

## 4.6 Job Management Module Implementation

The job management module enables employers to perform full CRUD (Create, Read, Update, Delete) operations on their job listings. Django's class-based views were used extensively in this module because they provide clean, reusable code for common operations.

The CreateJobView inherits from Django's CreateView and uses a custom JobForm that includes all required fields. A custom dispatch method is included in the view to verify that the requesting user is an authenticated employer before allowing access to the job creation form; if a non-employer attempts to access the URL, they are redirected to the home page with an error message.

The job listing is displayed on the public job search page as soon as it is created with the is_active field set to True. Employers can deactivate a listing (setting is_active to False) to remove it from public search results without permanently deleting it. A separate delete confirmation view prevents accidental deletion of job listings.

## 4.7 Job Search and Application Module Implementation

The job search functionality is one of the most important features of Fursad from a job seeker's perspective. The search view queries the database using Django's ORM with dynamic filter conditions that are built based on the user's search inputs.

The search functionality supports three types of filtering: keyword search (which uses Django's Q objects to search both the job title and description fields), location filter (which compares the job's location field with the selected value), and job type filter (which is an exact match on the job_type field). When multiple filters are combined, they are applied together using the AND operator, meaning that only jobs that match all specified criteria are returned.

The application submission process is handled by a single-click apply system. When a job seeker clicks the Apply Now button on a job detail page, an AJAX request is sent to the application endpoint, which creates a new JobApplication record linking the current user's JobSeekerProfile to the selected Job with a status of Pending. The system also checks whether the user has already applied for that

job before creating a new application, preventing duplicate applications. If the user has already applied, they are shown a message indicating this and the current status of their existing application.

## 4.8 Application Tracking Module Implementation

The application tracking module provides real-time visibility into the application process for both job seekers and employers. For job seekers, a dedicated My Applications page queries all JobApplication records where the applicant field matches the current user's JobSeekerProfile and displays them in a table sorted by date. Each row shows the job title, company name, date applied, and current status, with the status displayed using Bootstrap badge components in different colors (yellow for Pending, blue for Under Review, green for Accepted, and red for Rejected).

For employers, the Application Management view lists all applications received for a selected job listing, showing the applicant's name, their key skills, the date of application, and the current status. Employers can update the status of each application using a dropdown form, and the change is saved immediately to the database. The view also provides a link to view the applicant's full profile, allowing employers to review the complete resume and experience details before making a decision.

## 4.9 Administrative Interface

Django's built-in administration interface provides a powerful and ready-to-use dashboard for system administrators without requiring any additional development. The admin panel was configured by registering all of the project's models, which automatically generates a fully functional CRUD interface for each one.

Custom admin configurations were added to improve the usability of the admin interface. For example, the JobAdmin class was configured to display the job title, company name, location, job type, and active status in the list view, and to include search and filter capabilities. The JobApplicationAdmin was configured to show the applicant name, job title, application date, and status, making it easy to monitor application activity across the entire platform.

## 4.10 Security Implementation

Security was treated as a first-class concern throughout the implementation of Fursad. The following security measures were implemented:

- CSRF Protection: Django's CsrfViewMiddleware is enabled by default and was not disabled anywhere in the project. All forms include the csrf_token template tag, which renders a hidden input field containing the CSRF token. This prevents cross-site request forgery attacks.
- Input Validation and Sanitization: All user input is validated through Django's form system before being processed or saved to the database. The ORM automatically uses parameterized queries, preventing SQL injection attacks. All output rendered in templates is automatically HTML-escaped by Django's template engine, preventing XSS attacks.
- Authentication Decorators: Sensitive views are protected using the @login_required decorator, which redirects unauthenticated users to the login page. Custom permission checks verify the user's role (job seeker vs. employer) before allowing access to role-specific views.
- Secure File Upload Handling: Resume and profile photo uploads are validated for file type and size before being accepted. Files are stored outside the web root in the MEDIA directory and are served through Django's file serving mechanism rather than directly.
- Password Security: All passwords are hashed using Django's PBKDF2-SHA256 algorithm before storage. The application never stores or logs plain-text passwords. Password strength validation is enforced through Django's password validators.

## 4.11 Summary

This chapter has described the complete implementation of the Fursad job portal system. The development proceeded through four Agile sprints, following the design blueprint established in Chapter Three. Each major module, including user authentication, profile management, job posting, job search, application processing, application tracking, and the admin interface, was successfully

implemented using Django's robust framework features. Security was integrated throughout the development process using Django's built-in protections. The following chapter presents the testing strategy and results that validate the correctness and quality of the implemented system.

# CHAPTER FIVE: TESTING AND EVALUATION

## 5.1 Introduction

Testing is a critical phase in the software development life cycle that ensures the developed system meets all defined requirements, performs reliably under normal conditions, and handles error cases gracefully. This chapter presents the testing strategy applied to Fursad, including the types of tests performed, the test cases designed, the results obtained, and an overall evaluation of the system's quality. The testing process was conducted at multiple levels: unit testing for individual components, integration testing for combined modules, and user acceptance testing with real end users.

## 5.2 Testing Strategy

The testing strategy for Fursad follows a hierarchical approach, starting with the smallest units of code and progressing to full system testing with real users. This approach ensures that errors are caught at the earliest possible stage, where they are cheapest and easiest to fix. The following types of testing were performed:

- Unit Testing: Individual functions, views, and models were tested in isolation to verify that they perform their intended function correctly under a variety of input conditions.
- Integration Testing: The interactions between different modules were tested to verify that they work correctly when combined. For example, the integration between the user authentication module and the job application module was tested to ensure that only authenticated job seekers can submit applications.
- Functional Testing: Complete user workflows were tested end-to-end to verify that the system performs all required functions correctly from the user's perspective.
- Usability Testing: Representative users from both the job seeker and employer groups were asked to complete specific tasks using the system, and their experience was observed and documented.

- Security Testing: The system was tested for common web application vulnerabilities including SQL injection, XSS, and CSRF.

## 5.3 Unit Testing

Django provides a built-in testing framework based on Python's unittest module that was used for unit testing. Test cases were written for all major views, models, and forms. The tests were organized into separate test files within each application, following Django's testing conventions.

Key unit tests were written to verify: that the user registration forms correctly validate input and reject invalid data; that the login and logout views function correctly; that the job creation form correctly validates all required fields; that the job search view returns the correct results for various search and filter combinations; and that the application submission view correctly creates a JobApplication record and prevents duplicate applications.

A total of forty-three unit test cases were written. All forty-three tests passed successfully at the end of development, providing high confidence in the correctness of individual system components.

## 5.4 Integration Testing

Integration tests were designed to verify that the different modules of Fursad work correctly together. The following key integration scenarios were tested:

| ID | Integration Scenario | Expected Outcome | Result |
|---|---|---|---|
| IT-01 | Job seeker registers → completes profile → views job listings | User is created, profile is saved, job listings page displays correctly | PASS |
| IT-02 | Employer registers → posts job → job appears in search | Employer account created, job listing saved, job appears in public search results | PASS |
| IT-03 | Job seeker applies → employer sees application → updates status → seeker sees update | Application created with Pending status; employer sees it in management view; status update saved; seeker's tracking page shows new status | PASS |
| IT- | Unauthenticated user | User is redirected to login page | PASS |

| | | | |
|---|---|---|---|
| 04 | attempts to apply for a job | | |
| IT-05 | Job seeker attempts to access employer dashboard | User is redirected to home page with access denied message | PASS |
| IT-06 | Employer deactivates job listing | Job disappears from public search results immediately | PASS |
| IT-07 | Job seeker applies twice to the same job | Second application is rejected; user sees message that they have already applied | PASS |

*Table 5.1: Integration Test Cases and Results*

## 5.5 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was conducted with a group of eight real users: four job seekers (recent graduates) and four HR professionals acting as employers. Each participant was given a set of tasks to complete without assistance from the development team and was observed and timed. After completing the tasks, participants filled out a satisfaction questionnaire.

The tasks assigned to job seekers included: registering an account, completing a profile and uploading a resume, searching for jobs using specific keywords, applying for a job, and checking the status of an application. The tasks assigned to employers included: registering an employer account, completing a company profile, posting a new job listing, viewing received applications, and updating an application status.

| Evaluation Criteria | Excellent | Good | Fair | Poor |
|---|---|---|---|---|
| Ease of Registration | 6 (75%) | 2 (25%) | 0 | 0 |
| Clarity of Navigation | 5 (63%) | 3 (37%) | 0 | 0 |
| Job Search Effectiveness | 6 (75%) | 2 (25%) | 0 | 0 |
| Application Process Simplicity | 7 (88%) | 1 (12%) | 0 | 0 |
| Application Tracking Usefulness | 6 (75%) | 2 (25%) | 0 | 0 |
| Overall Satisfaction | 6 (75%) | 2 (25%) | 0 | 0 |

*Table 5.2: User Acceptance Testing Results (n=8 participants)*

The UAT results are strongly positive. Across all evaluation criteria, 100% of participants rated the system as either Good or Excellent, with the majority rating it Excellent. The application process received the highest satisfaction rating, with 88% of users rating it Excellent, confirming that the single-click apply feature met its design goal of maximum simplicity. No participants rated any aspect of the system as Poor, which indicates that the core functionality and usability goals have been successfully achieved.

Qualitative feedback from UAT participants highlighted several positive aspects of the system. Multiple participants commented on the clean and uncluttered interface, describing it as professional and easy to understand. Several employer participants expressed that the application management interface would be a significant improvement over their current paper-based processes. Job seeker participants were particularly positive about the application tracking feature, with one participant commenting that it was excellent to be able to see the status of my application in real time, as this was something I had never been able to do before.

Areas for improvement identified by UAT participants included a request for email notifications when application status changes, a desire to filter jobs by salary range, and a suggestion to add a direct messaging feature between employers and applicants. These features are noted as recommendations for future development in Chapter Six.

## 5.6 Security Testing

Basic security testing was performed to verify that the built-in Django security features are functioning correctly. The following tests were conducted:

- SQL Injection Test: Attempts were made to inject malicious SQL code through all form input fields and URL parameters. All injection attempts were blocked by Django's ORM parameterized query system, and the database remained unaffected.
- XSS Test: Attempts were made to inject JavaScript code through user-submitted content such as job descriptions and profile fields. Django's

automatic HTML escaping in templates prevented all injected scripts from executing.

- CSRF Test: Attempts were made to submit forms without a valid CSRF token by crafting requests from an external source. All such requests were rejected by Django's CsrfViewMiddleware with a 403 Forbidden response.
- Unauthorized Access Test: Attempts were made to directly access URLs that require authentication while logged out. All such attempts were correctly redirected to the login page. Attempts to access role-specific URLs (e.g., an employer trying to access job seeker pages) were correctly blocked.
- Password Security Test: Inspection of the database confirmed that all stored passwords are in hashed form and that no plain-text passwords are stored anywhere in the system.

All security tests produced the expected results, confirming that the system implements appropriate protection against common web application vulnerabilities.

## 5.7 System Evaluation Against Requirements

The final evaluation of Fursad assessed each defined system requirement against the testing results to determine the degree to which each requirement has been met:

| Req. ID | Requirement Description | Status |
|---|---|---|
| FR-01 | Users can register as job seeker or employer. | **Fully Met** |
| FR-02 | Registered users can log in and log out securely. | **Fully Met** |
| FR-03 | Job seekers can create and edit profiles, including uploading a resume. | **Fully Met** |
| FR-04 | Employers can post, edit, and delete job listings. | **Fully Met** |
| FR-05 | Job seekers can search and filter job listings. | **Fully Met** |
| FR-06 | Job seekers can apply for jobs with a single click. | **Fully Met** |
| FR-07 | Job seekers can view the status of their submitted applications. | **Fully Met** |

| | | |
|---|---|---|
| FR-08 | Employers can view and manage received applications. | **Fully Met** |
| NFR-01 | Passwords hashed using PBKDF2-SHA256. | **Fully Met** |
| NFR-02 | System usable on screens 320px and above. | **Fully Met** |
| NFR-03 | Page load times under three seconds. | **Fully Met** |
| NFR-04 | System handles user errors gracefully with clear messages. | **Fully Met** |

*Table 5.3: Requirements Evaluation Summary*

As shown in Table 5.3, all twelve defined requirements (eight functional and four non-functional) have been fully met by the implemented system. This represents a complete achievement of the system's specifications.

## 5.8 Summary

This chapter has presented the comprehensive testing and evaluation of Fursad. The unit testing, integration testing, user acceptance testing, and security testing collectively demonstrate that the system is functional, reliable, usable, and secure. All forty-three unit tests passed, all seven integration scenarios were verified, UAT participants reported high levels of satisfaction, and all security tests produced the expected protective behavior. The system evaluation confirms that all twelve defined requirements have been fully met. These results provide strong evidence that Fursad is a high-quality system ready for deployment and further development.

# CHAPTER SIX: CONCLUSION AND RECOMMENDATIONS

## 6.1 Summary of Findings

This project set out to design, develop, and evaluate Fursad, a web-based job portal system aimed at addressing the recruitment challenges faced by both job seekers and employers in the local employment market. The research began with a thorough analysis of the problems inherent in traditional recruitment processes, including limited access to job information, inefficient paper-based applications, poor candidate management, and a lack of transparency in the hiring process. These problems were confirmed through primary data collection including interviews with HR professionals and surveys of job seekers.

A comprehensive literature review established the theoretical and technical foundation for the project, confirming that digital recruitment platforms are highly effective at reducing recruitment costs and improving matching efficiency, and that Django is a proven, well-supported, and secure framework for building such systems. The review also confirmed that no existing platform adequately serves the specific needs of the local employment market.

The system was designed using the Django MVT architecture with a carefully planned data model, modular structure, and responsive user interface. The implementation was carried out over four Agile sprints, each delivering a functional increment of the system. The completed system was tested comprehensively, and all twelve defined requirements were confirmed as fully met. UAT results showed 100% satisfaction among participants, with the majority rating the system as Excellent across all evaluation dimensions.

## 6.2 Achievement of Objectives

Looking back at the eight research objectives defined in Chapter One, this project has successfully achieved all of them:

14. A comprehensive literature review was conducted, covering online recruitment systems, existing platforms, web development frameworks, and security considerations.
15. System requirements were defined based on primary data collection from HR professionals and job seekers, ensuring that the system is grounded in real user needs.
16. A robust system architecture was designed using Django's MVT pattern, with a well-structured relational database schema using SQLite3.
17. All core modules were fully implemented including user authentication, profile management, job posting, job search and filtering, application submission, application tracking, and administrative management.
18. A responsive user interface was developed using Bootstrap 5 and JavaScript, confirmed as fully functional on both desktop and mobile screens.
19. Comprehensive security measures were integrated, including CSRF protection, SQL injection prevention, XSS prevention, role-based access control, and PBKDF2-SHA256 password hashing.
20. Systematic testing was performed at unit, integration, functional, usability, and security levels, with all tests producing successful results.
21. The system was evaluated against all defined requirements, confirming full compliance, and evidence-based recommendations for future enhancements have been provided.

## 6.3 Contributions of the Project

This project makes several important contributions at different levels. At the practical level, Fursad delivers a fully functional, deployable web application that directly addresses real-world recruitment challenges in the local market. The system provides immediate, tangible value to job seekers who gain easier access to employment opportunities and to employers who gain a more efficient recruitment process.

At the technical level, the project demonstrates how the Django web framework can be effectively applied to build a secure, multi-role, data-driven web application within the constraints of an academic project. The complete design, implementation, and testing process documented in this thesis provides a valuable technical reference for future developers working on similar systems.

At the academic level, the project contributes to the growing body of knowledge on technology-driven solutions to employment challenges in developing regions. It demonstrates that modern software engineering practices and technologies can be successfully applied to solve local problems with significant social and economic impact.

## 6.4 Limitations of the Current System

While Fursad successfully meets all defined requirements, it is important to acknowledge the following limitations of the current version:

- Database Scalability: The use of SQLite3 as the database backend is appropriate for development and small-to-medium scale deployment but would need to be replaced with PostgreSQL or MySQL for large-scale production use.
- No Email Notifications: The current system does not send automatic email notifications to users when their application status changes, a feature that UAT participants identified as an important missing element.
- No AI-Based Matching: The system does not yet include intelligent job recommendation or candidate matching features. All matching is based on manual keyword search.
- No Native Mobile Application: While the web application is fully responsive and usable on mobile browsers, a dedicated native application for iOS and Android would improve the mobile user experience.
- Limited Testing Scale: The performance of the system under very high concurrent user loads has not been fully tested in this academic context.

## 6.5 Recommendations for Future Work

Based on the findings of this project and the feedback received from UAT participants, the following recommendations are made for the future development of Fursad:

22. Email Notification System: Implement automatic email notifications using Django's email backend (configured to use an SMTP provider such as SendGrid or Mailgun) to notify job seekers when their application status changes and to notify employers when they receive new applications.
23. AI-Powered Job Matching and Recommendations: Integrate a recommendation engine that analyzes a job seeker's skills, experience, and location to suggest the most relevant job listings. This could be implemented using Python's scikit-learn library or a more advanced natural language processing approach.
24. Database Migration to PostgreSQL: Migrate the database backend from SQLite3 to PostgreSQL to improve performance, concurrency handling, and scalability for larger user bases.
25. Native Mobile Application: Develop a mobile application for iOS and Android using React Native or Flutter, with a Django REST Framework API backend providing data services to the mobile client.
26. Real-Time Messaging: Add a real-time messaging feature using Django Channels and WebSockets to allow direct communication between employers and applicants within the platform.
27. Advanced Analytics Dashboard: Develop an analytics dashboard for administrators that provides insights into platform usage, job market trends, most in-demand skills, and employer activity.
28. Social Authentication: Implement social login options using OAuth 2.0 to allow users to register and log in using their Google or LinkedIn accounts.
29. Resume Parsing: Integrate an automated resume parsing library that extracts key information (skills, experience, education) from uploaded resume files and automatically populates the job seeker's profile fields.

## 6.6 Concluding Remarks

This graduation project has demonstrated that it is both technically feasible and practically valuable to develop a dedicated web-based job portal system for a local employment market using modern, open-source web development technologies. Fursad represents a significant step forward from the informal, fragmented, and inefficient recruitment methods that currently characterize the local employment market.

The project has shown that a small but dedicated development team, equipped with the right tools and a clear understanding of user needs, can build a system of genuine quality and practical utility within the time and resource constraints of an academic graduation project. The positive results of user acceptance testing confirm that the system is not merely a theoretical exercise but a practical tool that real people find useful and well-designed.

The limitations identified and the recommendations for future work provide a clear and actionable roadmap for the continued development and deployment of Fursad. The authors look forward to seeing the system grow beyond its current form to become a meaningful contributor to employment and economic development in the region.

In conclusion, this project demonstrates that thoughtful software design, rigorous engineering practices, and a genuine commitment to understanding and serving user needs can produce systems that make a real and positive difference in people's lives.

# REFERENCES

Al-Rababah, B. A., & Abu-Shanab, E. A. (2010). E-recruitment: Towards a deeper understanding of system usage. In Proceedings of the 2010 6th International Conference on Information Technology and Applications (ICITA) (pp. 1-5). IEEE.

Chun Tie, Y., Birks, M., & Francis, K. (2019). Grounded theory research: A design framework for novice researchers. SAGE Open Medicine, 7, 1-8. https://doi.org/10.1177/2050312118822927

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation). University of California, Irvine.

Freeman, A., & Robson, E. (2020). Head first design patterns: Building extensible and maintainable object-oriented software (2nd ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Holovaty, A., & Kaplan-Moss, J. (2009). The definitive guide to Django: Web development done right (2nd ed.). Apress.

Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Ndubuisi, N. O. (2020). Digital transformation of the recruitment process: An empirical study. Journal of Human Resource Management and Labour Studies, 8(1), 44-59.

Nielsen, J., & Budiu, R. (2013). Mobile usability. New Riders.

OWASP Foundation. (2021). OWASP top ten web application security risks. Open Web Application Security Project. https://owasp.org/www-project-top-ten/

Patel, N., & Shah, M. (2018). Security threats and mitigation strategies in web applications. International Journal of Computer Applications, 180(5), 18-24. https://doi.org/10.5120/ijca2018916386

Pressman, R. S., & Maxim, B. R. (2019). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum, the rules of the game. Scrum.org. https://scrumguides.org/

Singh, R., & Gupta, V. (2019). A systematic review of online recruitment systems. International Journal of Information Management, 46, 272-281. https://doi.org/10.1016/j.ijinfomgt.2018.08.011

Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.

Vincent, W. S. (2022). Django for professionals: Production websites with Python & Django (4.0 ed.). LearnDjango.com.

White, M. C. (2021). Modern web development with Django and REST APIs. Packt Publishing.

# APPENDICES

## Appendix A: User Survey Questionnaire

The following questions were included in the survey distributed to job seekers:

30. How do you currently search for job opportunities? (Multiple choice: Newspaper / Personal network / Social media / Company websites / Other)
31. How often do you actively search for new job opportunities? (Daily / Weekly / Monthly / Rarely)
32. What are the biggest difficulties you face when looking for a job? (Open text response)
33. Have you ever applied for a job through an online platform? (Yes / No)
34. How important is it for you to be able to track the status of your job applications online? (Scale 1-5)
35. What features would be most important to you in a job portal system? (Ranked list)
36. How comfortable are you with using websites and mobile applications? (Scale 1-5)
37. What type of device do you primarily use to access the internet? (Desktop / Laptop / Smartphone / Tablet)
38. Would you be willing to use an online job portal system if one were available locally? (Yes / No / Maybe)
39. Any additional comments or suggestions? (Open text response)

## Appendix B: System Requirements Specification

The following hardware and software requirements are necessary to deploy and run Fursad:

### *B.1 Minimum Server Requirements*

- Operating System: Ubuntu 20.04 LTS or later (Linux recommended); Windows Server 2019 also supported
- Processor: Dual-core CPU, 2.0 GHz or faster
- Memory: 2 GB RAM minimum (4 GB recommended)
- Storage: 20 GB available disk space (for application, database, and media files)
- Network: Stable internet connection for hosting

### *B.2 Software Requirements*

- Python 3.8 or higher
- Django 4.x (Long Term Support release recommended)
- pip (Python package manager)
- Git (for version control and deployment)
- A web server (Nginx or Apache) for production deployment
- Gunicorn or uWSGI as the WSGI application server

### *B.3 Client Requirements*

- Any modern web browser: Google Chrome 90+, Mozilla Firefox 88+, Safari 14+, or Microsoft Edge 90+
- JavaScript must be enabled in the browser
- Minimum screen resolution: 320px width (fully responsive design)
- Internet connection (broadband recommended for file uploads)

## Appendix C: Glossary of Technical Terms

| Term | Definition |
|---|---|
| API | Application Programming Interface: a set of rules and protocols that allows different software applications to communicate with each other. |
| ATS | Applicant Tracking System: software used by employers to |

| | |
|---|---|
| | electronically handle the recruitment process. |
| Bootstrap | A free, open-source CSS framework used for building responsive and mobile-friendly web interfaces. |
| CSRF | Cross-Site Request Forgery: a type of web security attack that tricks a user into performing unintended actions on a trusted website. |
| Django | A high-level Python web framework that follows the Model-View-Template architectural pattern. |
| MVT | Model-View-Template: the architectural pattern used by Django that separates data (Model), business logic (View), and presentation (Template). |
| ORM | Object-Relational Mapper: a programming technique that allows interaction with a relational database using object-oriented code instead of writing raw SQL. |
| SDLC | Software Development Life Cycle: a structured process used for planning, creating, testing, and deploying information systems. |
| SQLite3 | A lightweight, serverless relational database management system stored in a single file, widely used in embedded and small-to-medium web applications. |
| UAT | User Acceptance Testing: the final phase of testing in which actual users verify that the system meets their requirements. |
| XSS | Cross-Site Scripting: a type of web security attack in which malicious scripts are injected into web pages viewed by other users. |

*Table C.1: Glossary of Technical Terms*